

Iterative Statements

Introduction to Programming

StayAhead Training; Sep, 2024

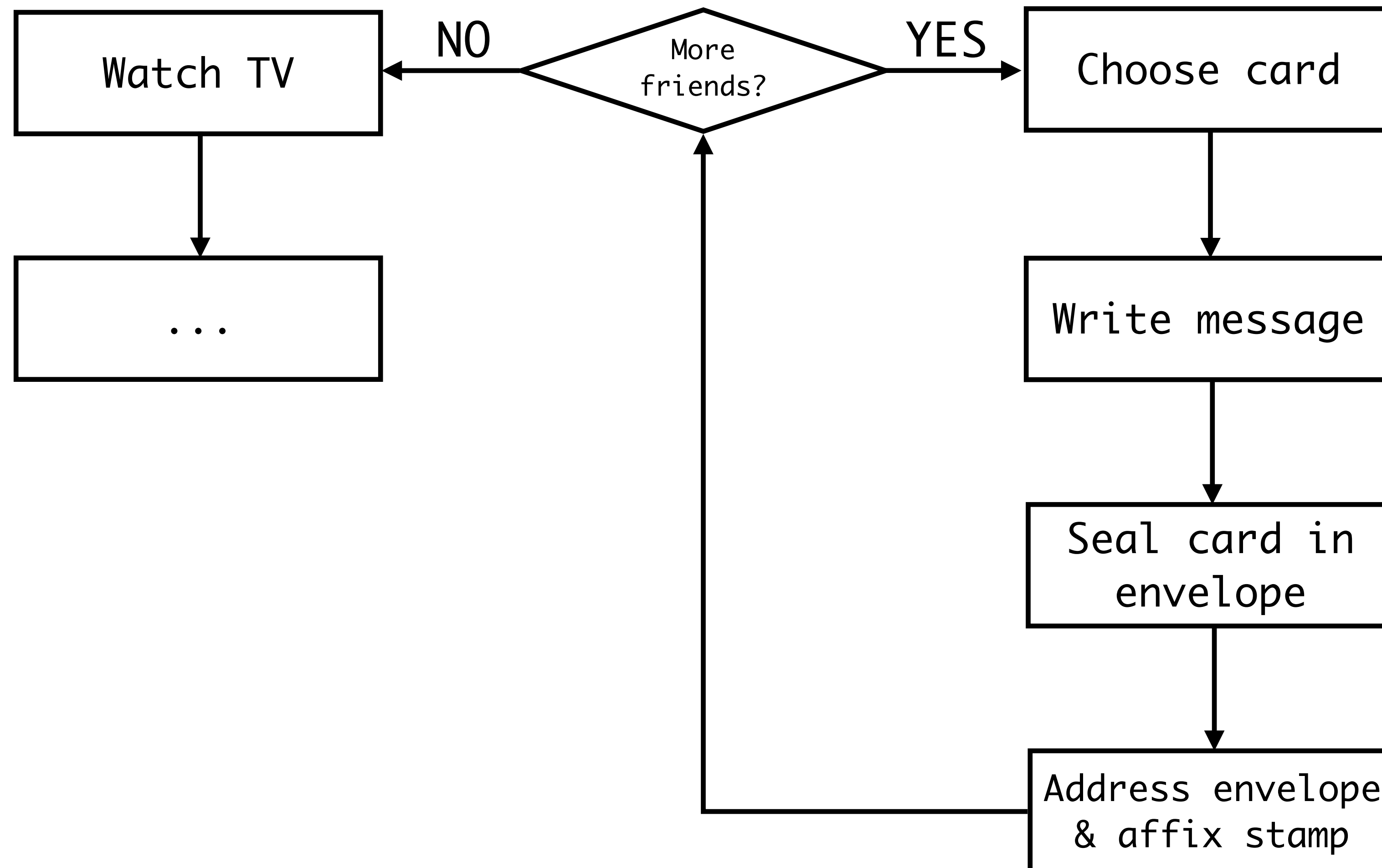
Introduction

Iterative Statements

- **Iterative statement:** one in which the assoc. block is executed n times
- Recall that a block is a group of statements that may or may not be executed

Introduction

Iterative Statements



While Loop

Iterative Statements

- Pseudocode, e.g.:
 - SET Count to 1
 - WHILE Count $\leq$ 3
 - OUTPUT Count to stdout
 - SET Count to Count + 1
 - ENDWHILE

While Loop

Iterative Statements

- PY: `x = 1`
- PY: `while x < 6:`
- PY: `print(x)`
- PY: `x += 1`

While Loop

Iterative Statements

- JS: `var x = 1;`
- JS: `while (x < 6) {`
- JS: `console.log(x);`
- JS: `x += 1;`
- JS: `}`

While Loop

Iterative Statements

- JV: `int x = 1;`
- JV: `while (x < 6) {`
- JV: `System.out.println(x);`
- JV: `x += 1;`
- JV: `}`

While Loop

Iterative Statements

- C#: `int x = 1;`
- C#: `while (x < 6) {`
- C#: `Console.WriteLine(x);`
- C#: `x += 1;`
- C#: `}`

For Loop

Iterative Statements

- The traditional for loop is shorthand for a while loop
- Each comprises three parts:
 1. Control variable initialisation (before first iteration)
 2. Boolean expression (at the start of each iteration)
 3. Control variable reassignment (at the end of each iteration)
- For loops are, arguably, harder to reason about than are while loops

For Loop

Iterative Statements

- PY: `for x in range(1, 6):`
- PY: `print(x)`

For Loop

Iterative Statements

- JS: `for (var x = 0; x < 6; x++) {`
- JS: `console.log(x);`
- JS: `}`

For Loop

Iterative Statements

- JV: `for (int x = 0; x < 6; x++) {`
- JV: `System.out.println(x);`
- JV: `}`

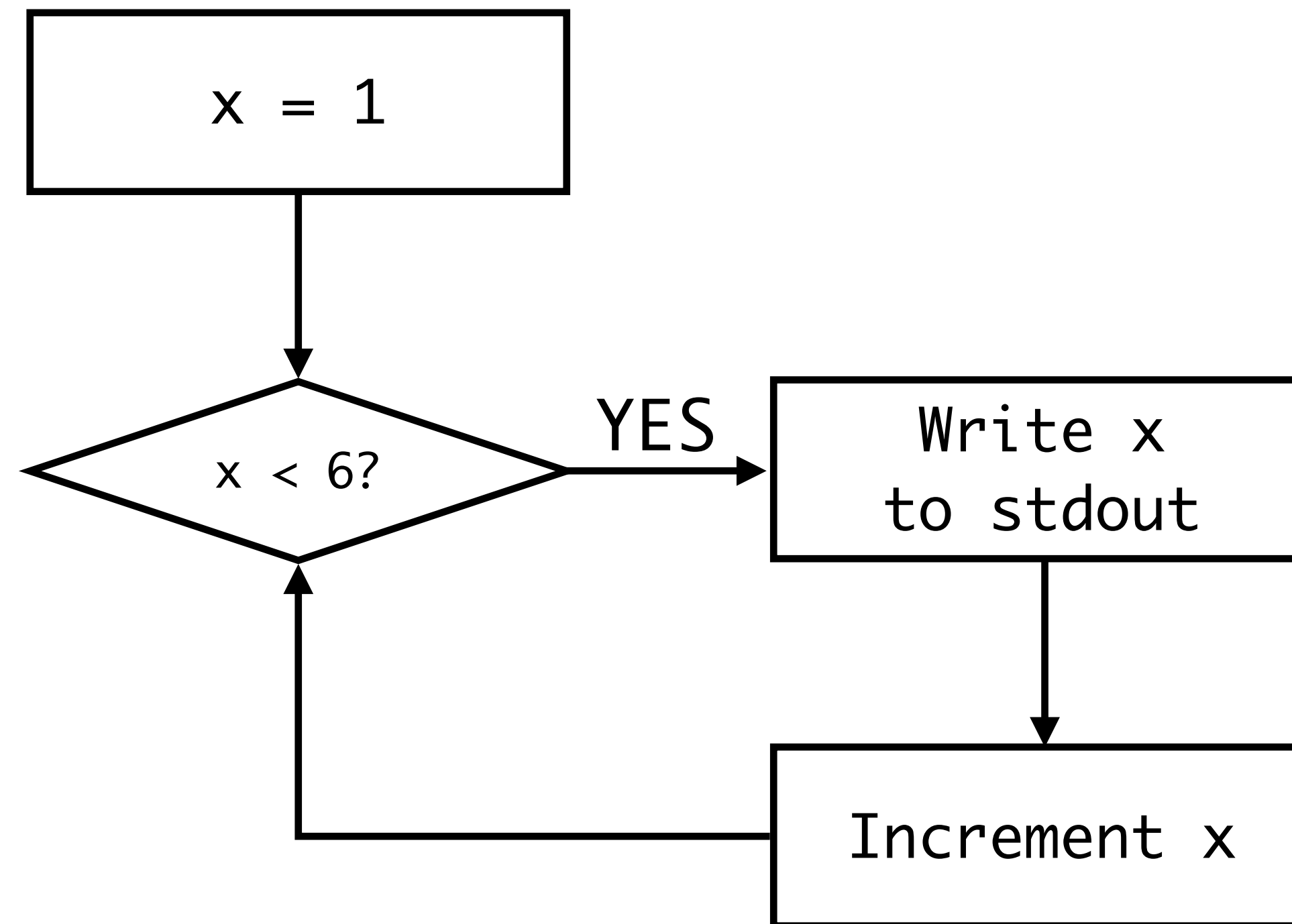
For Loop

Iterative Statements

- C#: `for (int x = 0; x < 6; x++) {`
- C#: `Console.WriteLine(x);`
- C#: `}`

For Loop

Iterative Statements



For Loop

Iterative Statements

Iteration #	Value of x at start	x < 6?	Value of x at end
1	1	True	2
2	2	True	3
3	3	True	4
4	4	True	5
5	5	True	6
6	6	False	

Code Blocks (again)

Iterative Statements

- PY: `for x in range(1, 6):`
- PY: `# instruction to be repeated code block`
- PY: `print(x)`

Traversing an Array

Iterative Statements

- Pseudocode, e.g.:
 - FOREACH Name in Names
 - OUTPUT Name to stdout
 - ENDFOREACH

Traversing an Array

Iterative Statements

- PY: `for` county `in` counties:
- PY: `print(county)`

Traversing an Array

Iterative Statements

- JS: `for (var county of counties) {`
- JS: `console.log(county);`
- JS: `}`

Traversing an Array

Iterative Statements

- JV: `for` (String county : counties) {
- JV: `System.out.println(county);`
- JV: }

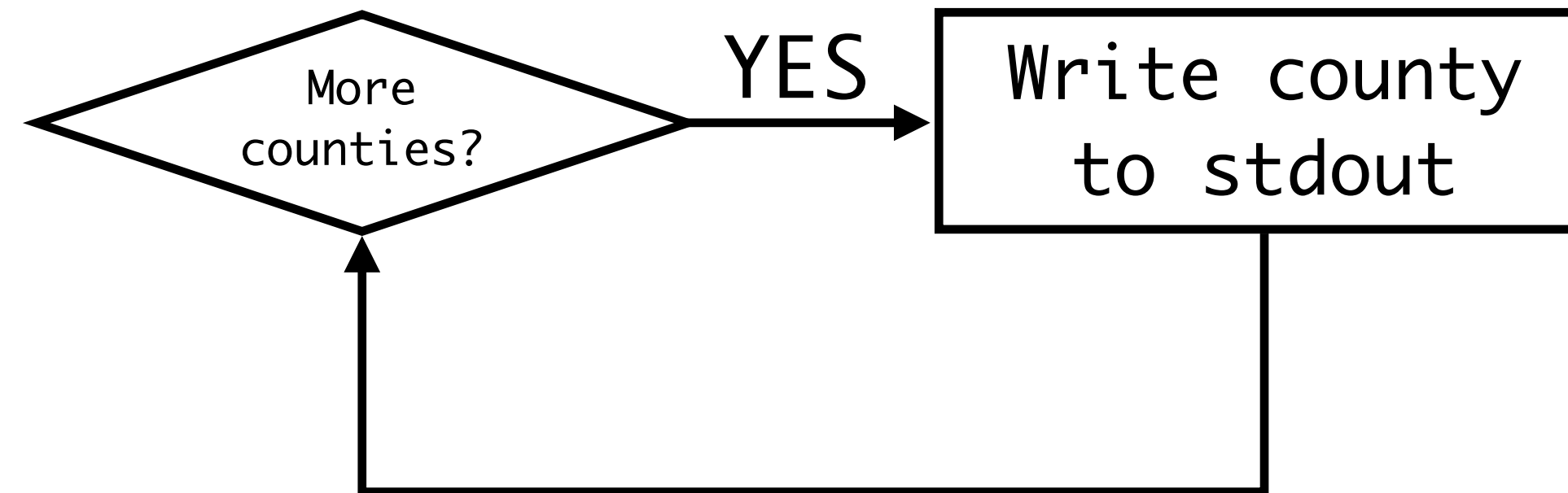
Traversing an Array

Iterative Statements

- C#: `foreach (string county in counties) {`
- C#: `Console.WriteLine(county);`
- C#: `}`

Traversing an Array

Iterative Statements



Break

Iterative Statements

- PY: `x = 1`
- PY: `while True:`
- PY: `if x == 6:`
- PY: `break`
- PY: `print(x)`
- PY: `x += 1`

Break

Iterative Statements

- JS: `var x = 1;`
- JS: `while (true) {`
- JS: `if (x == 6)`
- JS: `break;`
- JS: `console.log(x);`
- JS: `x += 1;`
- JS: `}`

Break

Iterative Statements

- JV: `int x = 1;`
- JV: `while (true) {`
- JV: `if (x == 6)`
- JV: `break;`
- JV: `System.out.println(x);`
- JV: `x += 1;`
- JV: `}`

Break

Iterative Statements

- C#: `int x = 1;`
- C#: `while (true) {`
- C#: `if (x == 6)`
- C#: `break;`
- C#: `Console.WriteLine(x);`
- C#: `x += 1;`
- C#: `}`

Continue

Iterative Statements

- PY: `x = 1`
- PY: `while x < 6:`
- PY: `if x == 3:`
- PY: `x += 1`
- PY: `continue`
- PY: `print(x)`
- PY: `x += 1`